

Algernon

Feature breakdown, architecture, and schedule

Game Algernon — a maze game about a mouse, after Flowers for Algernon

Platform Unity (C#), PC build

Team 7 developers — TL1, TL2, TL2+, TL3, TL4, TL5, TL6

Budget ~52 hours each · 364 developer-hours total

Milestones Coding Weeks 4-12 · Oral exam Week 13 · Demo Week 15

Generated from the TL5 AI Planning Prompt. Section A supplied by the team at the Meet and Greet; Section B is fixed course requirements. Each developer writes their Champion document from their own entry in Section 3.

CONTENTS

Before you read the feature specs: three things the team must settle first	3
1. Feature decomposition	4
2. Assignment of features to developers	5
3. Feature specifications	6
At a glance	6
F1 — Game Session & Level Flow (TL1)	7
F2 — Maze Generation & Level Data (TL2) · MVP core	8
F3 — Puzzle & Objective System (TL2+)	9
F4 — Hazards & Threats (TL3)	10
F5 — Perception, Camera & Visual Distortion (TL4)	11
F6 — HUD, Menus & Audio (TL5)	12
F7 — Player Character & Input (TL6)	13
4. Architectural overview	14
5. MVP core	15
6. How the features interact — coupling, cohesion, and GRASP	16
The architectural spine	16
GRASP, principle by principle	16
The honest weaknesses	16
7. Initial Gantt chart — Weeks 4 to 12	17
8. Risks and mitigations	18
Appendix — what to do next	19

Before you read the feature specs: three things the team must settle first

These come out of the constraints in the prompt itself. They are not optional housekeeping — each one will block someone in Week 4 if it is not resolved.

1. Where do shared interfaces live? Section 4 forbids shared implementation code and forbids modifying another developer's directories. But every feature below needs to see a small number of *interfaces and enums* (`IMazeQuery`, `DeathCause`, `DistortionId`, `LevelConfig`). The proposal in this plan is a `src/TL1_name/Contracts/` folder containing **interfaces, enums, and plain value structs only — zero implementation, zero MonoBehaviour**. It is owned by TL1; anyone needing a change opens an issue rather than editing it. This respects the letter of the rule (no shared *implementation*), but get explicit instructor sign-off on it before Week 4, because the whole architecture rests on it.

2. The MVP deadline and the coding gate collide. Section 6 puts the MVP at the end of Week 3. Gate G2 (Week 4) says coding may begin once Champions and the RFP are approved. TL1 cannot integrate seven features that are not allowed to exist yet. The workable reading: the Week 3 MVP is the *TL2 capstone platformer* deliverable plus a bare Unity scene with seven empty stub classes, and real feature code starts Week 4. Confirm this reading with the instructor — do not discover it in Week 3.

3. The "mirrored map" idea spans two features. Mirrored *controls* (input transformation) is TL6's, mandated by Section A5. Mirrored *rendering* (what you see) is TL4's. They are different code in different directories that happen to produce a similar player experience. TL1 decides, per level, which combination is active. Write this down in both Champion documents.

1. Feature decomposition

Seven features, one per developer, each ~52 hours, each independently testable.

#	Feature	One-line scope
F1	Game Session & Level Flow	Run lifecycle, state machine, timer, win/loss arbitration, integration hub
F2	Maze Generation & Level Data	Procedural mazes, grid model, seeded determinism, anchor points
F3	Puzzle & Objective System	Interactive puzzles, objective tree, exit unlocking
F4	Hazards & Threats	Traps, flooding, and the predator that hunts the mouse
F5	Perception, Camera & Visual Distortion	Camera rig, fog of war, mirrored/rotated/decaying view
F6	HUD, Menus & Audio	Timer readout, objectives, menus, result screens, sound
F7	Player Character & Input	The mouse: WASD movement, collision, control distortion

The split follows the game's own structure. *Algernon* is a maze, a mouse, a set of things to solve, a set of things that kill you, a way of seeing, a way of hearing about it, and a loop that decides when you've won. Each of those is one cohesive responsibility with a small interface at its edge, which is what makes the ownership rules in Section 4 survivable.

Two of the pre-agreed features from A4 are absorbed rather than standalone: random maze generation is the whole of F2, WASD movement is inside F7, and "puzzles that can kill the mouse" is deliberately split — the *solving* is F3, the *killing* is F4, because a lever that opens a door and a guillotine that falls on you have almost no code in common. The optional second-player controller lives in F7 as an explicitly optional deliverable, gated behind TL6's mandated work.

2. Assignment of features to developers

Developer	Feature	Why this developer
TL1	F1 — Game Session & Level Flow	Section 6 names TL1 the Initial Project Lead responsible for the MVP. The session loop <i>is</i> the integration seam, so the person who has to make seven features run together should own the object that makes them run together. Any other assignment would force TL1 to integrate through code they don't own.
TL2	F2 — Maze Generation & Level Data	Mandated by A5: one of the TL2s owns level generation as the MVP core. It is also the correct call independently — it is the feature with the most downstream consumers and the shortest interface, so it should belong to a developer with a Week 3/4 head start from the Unity tutorial lectures.
TL2+	F3 — Puzzle & Objective System	The puzzle system is F2's closest neighbour: it consumes maze anchor points and nothing else. Putting it with the second TL2 lets the two of them co-design the anchor contract in Week 4 while still working in entirely separate directories.
TL3	F4 — Hazards & Threats	The heaviest algorithmic feature — a State-machine predator with pathfinding, plus timed trap scheduling. It is also the most self-contained: it needs one <code>Vector2Int</code> pushed in per frame and publishes one event. Good fit for a developer who can go deep without waiting on anyone.
TL4	F5 — Perception, Camera & Visual Distortion	TL4 delivers the GRASP lecture in Week 7 and runs the Roast. F5 is the cleanest illustration of Pure Fabrication and Protected Variations in the codebase, which makes it a useful worked example to teach from. It is also almost fully independent — the camera needs a transform to follow and nothing more.
TL5	F6 — HUD, Menus & Audio	TL5 owns this planning prompt, the Champion process, and Week 3 coordination. F6 has a wide but shallow surface — many small widgets, each cheap — which absorbs interruption better than a feature with one deep algorithm. It is also the feature that can slip a week without blocking anyone else.
TL6	F7 — Player Character & Input	Mandated by A5: TL6 must own player movement and controls. Consistent with Section 1, which already assigns TL6 the later WebGL and mobile builds — the input abstraction layer they build in Week 5 is exactly what makes those ports possible.

3. Feature specifications

Each specification below is the raw material for that developer's Champion document. It contains every item required by Section 8.

At a glance

Dev	Feature	GOF pattern	What the dynamic binding demo shows
TL1	Game Session & Level Flow	Mediator (<i>large</i>)	<code>LevelRule.Evaluate</code> → the level becomes unwinnable and unloseable
TL2	Maze Generation & Level Data	Strategy + Private Class Data (<i>small</i> ×2)	<code>PostProcess</code> → the braided maze reverts to dead ends
TL2+	Puzzle & Objective System	Composite (<i>large</i>)	<code>IsSatisfied</code> → every puzzle completes on first touch
TL3	Hazards & Threats	State (<i>large</i>)	<code>Advance</code> → the cat freezes mid-patrol, the water stops rising
TL4	Perception, Camera & Visual Distortion	Chain of Responsibility (<i>large</i>)	<code>Process</code> → the mirrored level renders the right way round
TL5	HUD, Menus & Audio	Façade + Singleton (<i>small</i> ×2)	<code>Render</code> → the timer stops counting on screen
TL6	Player Character & Input	Decorator (<i>large</i>)	<code>Transform</code> → D moves right on the mirrored level

Every feature also carries a non-MonoBehaviour class hierarchy, one or more documented public methods, and an independent test path that needs no other feature to be complete.

F1 — Game Session & Level Flow (TL1)

Purpose. Owns the answer to "what is happening right now": which level is loaded, how much time is left, whether the run has been won or lost, and what happens next. Every other feature is a service; this is the thing that uses them. It is also the only place in the codebase where two features' names appear in the same file.

Responsibilities

- Session state machine: Boot → Menu → LevelLoading → Playing → Paused → Won → Lost → RunComplete
- Level progression and per-level configuration (maze size, hazard density, active distortions, time limit)
- The countdown timer and the time-out loss condition
- Per-frame orchestration: push the player's cell to F3 and F4, poll objective completion, arbitrate death
- Run statistics: elapsed time, deaths, puzzles solved, levels cleared
- Scene bootstrapping and initialisation ordering
- Custodian of `Contracts/` — the interface and enum definitions every feature compiles against

Public interface

```
void StartRun(int startingLevelIndex);
void RestartLevel();
void AdvanceLevel();
void Pause(bool paused);
void AbortRun();
SessionState State { get; }           // enum
LevelConfig CurrentLevel { get; }     // immutable value object
RunStatistics Statistics { get; }
event Action<LevelConfig> OnLevelStarted;
event Action<LevelResult> OnLevelEnded; // carries win/loss + cause
event Action<RunStatistics> OnRunEnded;
```

Internal classes. `GameCoordinator` (MonoBehaviour, thin — forwards `Update` and nothing else), `SessionMachine`, `SessionState`, `LevelCatalogue`, `LevelConfig`, `RunTimer`, `RunStatistics`, `SessionSnapshot`, and the `LevelRule` hierarchy below.

Proposed class hierarchy (non-MonoBehaviour).

```
abstract LevelRule
├─ PuzzleCompletionRule // win when F3 reports all objectives complete
├─ TimedSurvivalRule    // lose when RunTimer hits zero
├─ NoDeathRule          // lose on the first death event from F4
└─ ReachExitRule        // win when player cell == maze exit cell
```

Base method `public virtual RuleVerdict Evaluate(SessionSnapshot snapshot)` returns `RuleVerdict.InProgress` unconditionally. Each subclass overrides it. `SessionMachine` holds `List<LevelRule>` and evaluates every rule each frame through the base-typed reference — which is what makes the dynamic binding real rather than decorative.

Dynamic binding demo. Change `virtual` to nothing on `LevelRule.Evaluate` and `override` to `new` on the subclasses. The list is `List<LevelRule>`, so every call now resolves to the base method, every rule returns `InProgress`, and the level becomes unwinnable and unloseable — reach the exit and nothing happens, run the timer to zero and nothing happens. Large and obvious on screen.

Proposed GOF pattern — Mediator (large). `GameCoordinator` is a textbook Mediator: F2 through F7 never hold references to each other, only to the `Contracts` interfaces, and all cross-feature communication routes through the coordinator. Chosen over letting features talk directly because Section 4 *requires* it — direct references would mean a developer's compile depends on another developer's concrete types, and integration would mean editing someone else's directory. The pattern is not decoration here; it is the mechanism that makes the ownership rules enforceable.

Proposed GRASP — Controller (primary: `GameCoordinator` is the system event handler for the run), with **Indirection** and **Low Coupling** as the consequences, and **Pure Fabrication** for `SessionMachine`, which represents nothing in the game's domain and exists purely to hold behaviour.

Dependencies. All six other features — by design, and one-directionally. TL1 depends on their interfaces; nobody depends on TL1's implementation. If any feature is missing, its interface is filled with a `NullX` stub from `Contracts/` and the game still runs, which is what Section 6 requires of the MVP.

Testing strategy. `SessionMachine` and every `LevelRule` are plain C# classes taking a `SessionSnapshot` struct — no Unity, no other feature, testable in isolation with hand-built snapshots. Boundary tests: timer at exactly 0.0s; a level with zero puzzles (should be winnable immediately on reaching the exit). Stress test: 10,000 state transitions with randomised snapshots, asserting the machine never enters an undefined state.

Completion criteria. A run of at least three levels can be started from the menu, played, won, lost, restarted and completed without leaving Play Mode; every feature is driven exclusively through its `Contracts` interface; the `LevelRule` dynamic binding demo runs live in under 60 seconds.

Effort (~52h). Design 10 · Coding 20 · Testing 12 · Documentation 10. Documentation is high because TL1 also writes the integration contract that the other six Champion documents reference.

F2 — Maze Generation & Level Data (TL2) · MVP core

Purpose. Produces the maze and owns the grid that everything else queries. This is the feature the game cannot exist without, and by A5 it is the minimum viable product.

Responsibilities

- The grid data model: cells, wall flags, bounds, coordinate conversion
- Generation algorithms and their selection by difficulty
- Seeded, reproducible generation — the same seed must always produce the same maze
- Spawn and exit placement, with a guaranteed path between them
- Anchor points: typed, evenly-distributed cells that F3 and F4 place content on, handed out without either feature needing to understand maze structure
- Building the Unity tilemap or wall geometry from the grid
- Read-only query API for walkability and neighbours

Public interface

```
MazeData Generate(MazeConfig config);          // config carries seed + difficulty
MazeData Current { get; }
bool IsWalkable(Vector2Int cell);
Vector2Int SpawnCell { get; }
Vector2Int ExitCell { get; }
Bounds WorldBounds { get; }
Vector3 CellToWorld(Vector2Int cell);
Vector2Int WorldToCell(Vector3 worldPosition);
IReadOnlyList<Vector2Int> ReserveAnchors(AnchorKind kind, int count);
bool PathExists(Vector2Int from, Vector2Int to);
void Clear();
```

`ReserveAnchors` is the important one: it hands out cells and remembers what it gave away, so F3's lever and F4's spike can never land on the same tile without either feature knowing the other exists.

Internal classes. `MazeGenerator` (thin `MonoBehaviour` façade), `MazeData`, `Cell`, `WallFlags`, `MazeConfig`, `AnchorRegistry`, `MazeRenderer`, and the `MazeAlgorithm` hierarchy.

Proposed class hierarchy (non-`MonoBehaviour`).

```
abstract MazeAlgorithm
├─ RecursiveBacktrackerAlgorithm  // long winding corridors, few branches
├─ PrimsAlgorithm                // short branchy corridors, many dead ends
└─ BraidedAlgorithm              // backtracker + dead-end removal → loops
```

Two virtual methods: `public virtual void Carve(MazeData grid, Random rng)` and `public virtual void PostProcess(MazeData grid, Random rng)`. The base `PostProcess` does nothing — the maze stays a perfect maze with dead ends. `BraidedAlgorithm` overrides it to knock through walls at dead ends, producing loops.

Dynamic binding demo. Drop `virtual` from `PostProcess` (and `override` → `new` on `BraidedAlgorithm`). `MazeData` is generated through an `MazeAlgorithm`-typed field, so the braiding step silently stops running: the maze on screen visibly changes from an open looping layout to a dead-end maze, in the same frame, with the same seed. Side-by-side screenshots make this an easy 30-second oral exam answer.

Proposed GOF patterns — two small: Strategy + Private Class Data. *Strategy* for `MazeAlgorithm` — the generator picks an algorithm per difficulty band and never branches on a type code. Chosen over an `if (difficulty > 5)` chain because new algorithms are the most likely requirement change in this feature, and each one is genuinely interchangeable. *Private Class Data* for `MazeConfig` — seed, dimensions and braiding ratio are set once in the constructor and exposed read-only, so no caller can mutate generation parameters mid-generation. Chosen because reproducibility is the one property every other developer's tests depend on; making it structurally impossible to break is worth a class.

Proposed GRASP — Information Expert (`MazeData` answers every question about the maze because it is the only object that holds the grid) and **Creator** (`MazeAlgorithm` instantiates `Cell` objects because it is the only thing that has the data to initialise them), with **Protected Variations** at the `IMazeQuery` interface.

Dependencies. None. F2 compiles and runs with no other feature present — the reason it is the MVP core.

Testing strategy. Fully headless. Generate with a fixed seed, then assert: BFS from spawn reaches exit; every reserved anchor is walkable; the same seed twice produces byte-identical grids. Boundary tests: minimum 5×5 maze; even-dimension maze (odd/even off-by-one is the classic bug here). Stress test: generate a 200×200 maze 100 times and assert no allocation spike and no generation exceeding 250 ms.

Completion criteria. Any difficulty level generates a solvable maze in under 100 ms, renders correctly, hands out non-colliding anchors, and reproduces exactly from a seed printed to the console.

Effort (~52h). Design 9 · Coding 22 · Testing 13 · Documentation 8. Testing is high because six other features trust this feature's guarantees.

F3 — Puzzle & Objective System (TL2+)

Purpose. The things the mouse must *solve*. Owns the win condition: the exit does not open until this feature says the level's objectives are complete.

Responsibilities

- The puzzle element hierarchy and each element's own completion logic
- The objective tree: which puzzles, in which combination, unlock the exit
- Placement of puzzle instances onto anchor points supplied by F2
- Progress reporting for the HUD
- Full reset on death or level restart — a partially-solved puzzle must not persist
- Per-difficulty puzzle composition (how many, which kinds, how they combine)

Public interface

```
void BuildPuzzles(PuzzleConfig config, IReadOnlyList<Vector2Int> anchors);
void NotifyPlayerCell(Vector2Int cell);      // pushed by TL1 each frame
void NotifyInteractPressed();               // pushed by TL1 on input
bool AllObjectivesComplete { get; }
PuzzleProgress Progress { get; }           // completed / total / label
string ObjectiveText { get; }
void ResetPuzzles();
event Action<PuzzleProgress> OnProgressChanged;
event Action OnExitUnlocked;
```

Note that F3 never sees the player object, only a cell coordinate pushed in by TL1. That single decision is what keeps this feature independently testable.

Internal classes. `PuzzleSystem` (thin `MonoBehaviour` façade), `ObjectiveTree`, `PuzzleContext`, `PuzzleProgress`, `PuzzleConfig`, `PuzzleFactory`, and the `PuzzleNode` hierarchy.

Proposed class hierarchy (non-`MonoBehaviour`).

```
abstract PuzzleNode
├─ CompositeNode      // holds children; AND / OR / SEQUENCE
└─ abstract PuzzleElement
   ├─ PressurePlateElement // complete while the mouse stands on it
   ├─ LeverElement        // toggled by interact, stays toggled
   ├─ KeyCollectionElement // complete when all N keys collected
   └─ PushBlockElement    // complete when block reaches target cell
```

Base public virtual bool `IsSatisfied(PuzzleContext ctx)` returns true as soon as the node's cell has been touched once. Every element overrides it with real logic.

Dynamic binding demo. Remove `virtual` from `IsSatisfied`. The `ObjectiveTree` walks `PuzzleNode` references, so every puzzle now completes on first touch: walk onto the pressure plate and the exit opens instantly, the lever sequence stops mattering, and the HUD counter jumps from 0/4 to 4/4. Highly visible and instantly understandable.

Proposed GOF pattern — Composite (large). `CompositeNode` and `PuzzleElement` share the `PuzzleNode` interface, so a level objective can be an arbitrary tree: "open the exit when (plate A AND plate B) OR the brass key is collected". Chosen over a flat `List<Puzzle>` with a counter because the later levels in A2 explicitly escalate difficulty through combination, and a flat list can only express AND. The tree also gives the recursive `IsSatisfied` call that makes the polymorphism demo above meaningful.

Proposed GRASP — Information Expert (each element evaluates its own completion because it is the only object holding its state) and **High Cohesion** (every class here is about one question: is this level solved yet?), with **Indirection** via `ObjectiveTree`, which keeps individual puzzles ignorant of the exit door.

Dependencies. Anchor cells from F2 at build time, and a player cell pushed in per frame by F1. No compile-time reference to either.

Testing strategy. Hand-build an `ObjectiveTree` in a test, feed synthetic cell coordinates, assert `AllObjectivesComplete` flips at the right moment. Boundary tests: a tree with a single element; a tree whose OR branch is satisfied before its AND branch is even placed. Stress test: 500 elements in a nested tree, evaluated every frame for 1,000 frames, asserting no per-frame allocation.

Completion criteria. Four distinct puzzle types work, compose into AND/OR/ SEQUENCE trees, drive the HUD counter, reset cleanly on death, and unlock the exit exactly once.

Effort (~52h). Design 9 · Coding 23 · Testing 12 · Documentation 8.

F4 — Hazards & Threats (TL3)

Purpose. The things that *kill* the mouse. Owns everything on the loss side of A2 except the clock, which belongs to F1.

Responsibilities

- The hazard hierarchy: static spikes, timed traps, rising water, glue, and a patrolling predator
- Hazard behaviour over time — patrol routes, trap cycles, flood advance
- Telegraphing: every lethal hazard warns before it fires, so deaths read as fair
- Threat evaluation against a pushed player cell, and publishing the death event
- Difficulty scaling: density, cycle speed, predator aggression
- Full reset on level restart

Public interface

```
void SpawnHazards(HazardConfig config, IReadOnlyList<Vector2Int> anchors);
void Tick(float deltaTime, Vector2Int playerCell); // pushed by TL1
void SetActive(bool active); // paused / level intro
void ResetHazards();
int ActiveHazardCount { get; }
IReadOnlyList<Vector2Int> ThreatenedCells { get; } // for HUD/minimap
event Action<DeathCause> OnPlayerCaught;
```

Internal classes. `HazardSystem` (thin `MonoBehaviour` façade), `HazardScheduler`, `HazardConfig`, `PredatorBrain`, `PatrolRoute`, `GridPathfinder`, the `Hazard` hierarchy, and the `PredatorState` hierarchy.

Proposed class hierarchy (non-`MonoBehaviour`).

```
abstract Hazard
├─ StaticSpikeHazard // always lethal on its cell
├─ TimedTrapHazard // lethal on a cycle, telegraphs 0.5s early
├─ FloodHazard // lethal region grows over time
└─ PredatorHazard // moves; owns a PredatorState machine
```

Base `public virtual void Advance(float deltaTime)` is empty — correct for a spike, which never changes. `TimedTrapHazard` overrides it to run its cycle, `FloodHazard` to expand its region, `PredatorHazard` to step its state machine.

Dynamic binding demo. Remove `virtual` from `Advance`. `HazardScheduler` iterates `List<Hazard>`, so every dynamic hazard freezes: the cat stops mid-patrol and never moves again, the guillotine hangs open forever, the water stops rising. The level becomes trivially survivable and the screen visibly stops moving. One of the strongest demos in the project.

Proposed GOF pattern — State (large). `PredatorHazard` delegates to a `PredatorState`: `PatrolState` → `AlertState` → `ChaseState` → `SearchState` → `ReturnState`. Each state handles its own transitions. Chosen over an enum plus a switch statement because the predator has five states with distinct entry and exit behaviour, and the switch version becomes unreadable at around three. It is also the pattern most likely to be asked about at the oral exam, and it has a live demo: stand still and watch the state label change in the inspector.

Proposed GRASP — Polymorphism (hazard variation handled by type, not by conditionals) and **Low Coupling** (F4 has no reference to the player, only a `Vector2Int`), with **Pure Fabrication** for `HazardScheduler` and `GridPathfinder`, neither of which is a domain concept.

Dependencies. Anchor cells from F2, walkability queries via `IMazeQuery` for pathfinding, and the player cell pushed in by F1. The kill itself is *arbitrated* by F1 — F4 raises `OnPlayerCaught` and F1 decides what that means, so F4 never calls into F7.

Testing strategy. Fully headless with a hand-built 10×10 stub maze implementing `IMazeQuery`. Assert the predator reaches a target cell in the expected number of steps; assert `OnPlayerCaught` fires exactly once per lethal frame, not every frame. Boundary tests: predator and player on the same cell at spawn; a maze with no valid patrol route. Stress test: 200 hazards ticked for 5,000 frames without frame-time regression.

Completion criteria. Four hazard types work, the predator demonstrates all five states in a single playthrough, every death is telegraphed, and hazards reset correctly on retry.

Effort (~52h). Design 9 · Coding 24 · Testing 11 · Documentation 8.

F5 — Perception, Camera & Visual Distortion (TL4)

Purpose. What the player can see. This is where the *Flowers for Algernon* theme lives mechanically — the view sharpens and decays, mirrors, and rotates as levels progress.

Responsibilities

- The camera rig: follow, zoom-to-fit, framing on level start, screen shake
- Fog of war: a vision radius around the mouse that reveals explored cells permanently
- The visual distortion pipeline: mirror, rotation, desaturation, blur, blackout
- The minimap, optionally distorted to match
- Per-level distortion scheduling and intensity ramping

Public interface

```
void BindFollowTarget(Transform target);
void FrameMaze(Bounds mazeBounds);
void ApplyDistortion(DistortionId id, float intensity);
void RemoveDistortion(DistortionId id);
void ClearDistortions();
void SetVisionRadius(float cells);
void RevealAll(bool reveal);           // win screen / debug
void Shake(float magnitude, float seconds);
CameraSnapshot CaptureState();        // for pause / restart
void RestoreState(CameraSnapshot snapshot);
```

Internal classes. `ViewDirector` (thin `MonoBehaviour` façade), `CameraRig`, `CameraState`, `ViewEffectPipeline`, `FogGrid`, `MinimapRenderer`, `CameraSnapshot`, and the `ViewEffect` hierarchy.

Proposed class hierarchy (non-`MonoBehaviour`).

```
abstract ViewEffect           // holds Next; may pass along or halt
├─ MirrorViewEffect           // flips the view horizontally
├─ RotationViewEffect         // rotates the view by N degrees
├─ FogViewEffect              // masks unexplored cells
├─ DesaturationViewEffect     // drains colour as the timer runs down
└─ BlackoutViewEffect         // halts the chain – nothing after it renders
```

Base `public virtual CameraState Process(CameraState state)` returns the state unchanged and forwards it to `Next`. Each effect overrides it.

Dynamic binding demo. Remove `virtual` from `Process`. The pipeline holds `ViewEffect` references, so every effect degrades to pass-through: the mirrored level renders the right way round, the fog lifts, the rotation stops. The screen changes completely while the game logic behaves identically — a particularly clean demonstration that the *runtime type* is what mattered.

Proposed GOF pattern — Chain of Responsibility (large). Effects form a chain; each may modify the `CameraState` and pass it on, **or stop the chain entirely** — `BlackoutViewEffect` does exactly that, because nothing downstream should render through a blackout. That halting behaviour is what makes this Chain of Responsibility rather than Decorator, and TL4 should be ready to say so out loud at the oral exam. Chosen over a fixed sequence of `if` blocks because levels combine effects in arbitrary orders, and order matters (rotate-then-mirror $\neq$ mirror-then-rotate).

Proposed GRASP — Pure Fabrication (`ViewEffectPipeline` and `FogGrid` are invented classes with no counterpart in the game world, created solely to keep rendering concerns out of the camera) and **Protected Variations** (F5's interface is stable while its rendering internals change freely), with **Low Coupling** — F5 knows only a `Transform`, a `Bounds`, and an enum.

Dependencies. A follow target `Transform` from F7 and maze bounds from F2, both handed over by F1 at level start. No per-frame dependency at all.

Testing strategy. `ViewEffect` classes are pure functions over `CameraState` — feed a known state through a chain and assert the output matrix and mask. Boundary tests: an empty chain (must be identity); a chain where `BlackoutViewEffect` is first (nothing downstream must execute — assert with a counting spy effect). Stress test: 20 stacked effects for 10,000 frames.

Completion criteria. Camera follows and frames correctly, fog reveals and persists, at least four distortions work individually and in combination, and the chain-halting blackout is demonstrable.

Effort (~52h). Design 10 · Coding 22 · Testing 12 · Documentation 8.

F6 — HUD, Menus & Audio (TL5)

Purpose. Everything the player reads and hears. Turns the state that F1 already knows into something a person can act on.

Responsibilities

- Main menu, level select, pause menu, settings
- In-run HUD: countdown timer, objective progress, attempts, active-distortion indicator
- Level intro and outro cards, win and loss screens with run statistics
- The audio layer: music, SFX cues, volume buses, and the mapping from game events to sounds
- A string table so all player-facing text lives in one place

Public interface

```
void ShowHud(bool visible);
void SetTimer(float secondsRemaining);
void SetObjective(string label, int completed, int total);
void SetDistortionIndicator(IReadOnlyList<DistortionId> active);
void ShowLevelIntro(int levelIndex, string title, Action onDismissed);
void ShowResult(LevelResult result, RunStatistics stats);
void ShowPause(bool paused);
void PlayCue(AudioCueId cue);
void SetMusicIntensity(float normalised); // ramps as the timer drops
event Action OnStartPressed;
event Action OnRetryPressed;
event Action OnQuitPressed;
```

Internal classes. `HudFacade` (thin `MonoBehaviour` façade), `HudModel`, `ScreenRouter`, `AudioDirector`, `AudioCueTable`, `StringTable`, and the `HudWidget` hierarchy.

Proposed class hierarchy (non-`MonoBehaviour`).

```
abstract HudWidget
├─ TimerWidget           // mm:ss, flashes red under 10 seconds
├─ ObjectiveWidget       // "Levers: 2/4"
├─ AttemptsWidget        // death counter
└─ DistortionWarningWidget // icons for active distortions
```

Base `public virtual string Render(HudModel model)` returns the widget's raw label with no formatting or state. Each widget overrides it.

Dynamic binding demo. Remove `virtual` from `Render`. `HudFacade` iterates `List<HudWidget>`, so the HUD collapses to four static placeholder labels: the timer stops counting down on screen even though the run is still timed, the objective counter freezes at its default, and the red low-time flash never fires. Easy to show and easy to explain.

Proposed GOF patterns — two small: Façade + Singleton. *Façade* — `HudFacade` is the single object F1 talks to; it hides a dozen widgets, three screens, and the audio director behind one interface. Chosen over exposing widgets individually because that would put F6's internal structure into F1's code and break the ownership boundary the moment a widget is renamed. *Singleton* — `AudioDirector`, because Unity permits exactly one `AudioListener` and audio cues are fired from screens, widgets and F1 alike; the alternative is threading a reference through every one of them. TL5 should be ready to acknowledge the usual Singleton criticisms and explain why the constraint is genuine here rather than convenient.

Proposed GRASP — Pure Fabrication (`ScreenRouter` and `HudModel` exist to hold behaviour, not to model anything in the game) and **High Cohesion** (one feature, one question: what does the player see and hear about the current state?), with **Indirection** — widgets read a `HudModel` and never query other features.

Dependencies. F1 only. Everything F6 displays arrives through the interface above. This makes F6 the most trivially stubable feature in the project and a good candidate to build against a fake `HudModel` from Week 4.

Testing strategy. `HudWidget.Render` returns a string, so every widget is a pure function testable without Unity: build a `HudModel`, assert the output text. Boundary tests: timer at exactly 10.0s (flash threshold), and 0/0 objectives (must not divide by zero or read "0/0"). Stress test: 60 `SetTimer` calls per second for 10 minutes with no allocation growth.

Completion criteria. All screens navigate correctly with keyboard and mouse, the HUD updates live during a run, results show accurate statistics, audio cues fire for at least eight distinct events, and no player-facing string is hard-coded outside `StringTable`.

Effort (~52h). Design 8 · Coding 24 · Testing 12 · Documentation 8.

F7 — Player Character & Input (TL6)

Purpose. The mouse. Movement, collision, animation, and — the interesting part — the control distortions that A2 names as a difficulty source in later levels.

Responsibilities

- WASD movement with collision against maze walls
- An input abstraction layer between the device and the character, so WebGL and mobile bindings can be added later without touching movement code
- The control transformation pipeline: mirrored, inverted, rotated, lagging, drifting controls
- Player state: alive, dead, frozen, spawning; animation state driven from it
- Position and cell queries for other features
- *Optional:* a second player controller for a co-op level (A4), gated behind everything above

Public interface

```
void SpawnAt(Vector3 worldPosition);
void Despawn();
Vector3 Position { get; }
Vector2Int CurrentCell { get; }
Transform FollowTarget { get; }           // handed to F5
bool IsAlive { get; }
void Kill(DeathCause cause);              // called by F1, not by F4
void SetControlStack(IReadOnlyList<ControlModifierId> modifiers);
void EnableInput(bool enabled);
void FreezeInput(float seconds);
event Action<Vector2Int> OnCellChanged;
event Action OnInteractPressed;
```

Internal classes. `PlayerController` (thin `MonoBehaviour` façade), `MouseMotor`, `GridCollisionResolver`, `PlayerStateMachine`, `RawInputSource`, and the `InputTransformer` hierarchy.

Proposed class hierarchy (non-`MonoBehaviour`).

```
IInputSource           // Vector2 Read()
├─ RawInputSource       // reads the device
├─ abstract InputTransformer // wraps an inner IInputSource
│   ├── MirrorTransformer // negates X
│   ├── InvertTransformer // negates Y
│   ├── RotationTransformer // rotates the vector by N degrees
│   └─ LagTransformer    // returns input from N frames ago
```

Base `public virtual Vector2 Transform(Vector2 raw)` returns `raw` unchanged. Each transformer overrides it. `MouseMotor` holds an `IInputSource` and never knows how many transformers are stacked behind it.

Dynamic binding demo. Remove `virtual` from `Transform`. The stack is composed through `InputTransformer` references, so every distortion becomes a pass-through: on the mirrored level, pressing D moves the mouse right instead of left, and the level that was disorienting becomes trivially easy. This is probably the single most legible dynamic-binding demo in the project — the change is in the player's hands, not just on screen.

Proposed GOF pattern — Decorator (large). Each `InputTransformer` wraps an `IInputSource` and is itself an `IInputSource`, so distortions stack at runtime: level 9 might be `Lag(Mirror(Raw))`. Chosen over a Strategy with one class per combination because the combinations multiply — four modifiers give sixteen combinations, and Decorator handles all of them with four classes. Chosen over a flat list of enums with a switch because order matters: `Rotate(Mirror(x))` and `Mirror(Rotate(x))` are different, and composition makes that explicit rather than accidental.

Proposed GRASP — Protected Variations (the `IInputSource` seam is exactly what lets TL6 add touch and WebGL bindings later without any movement code changing, per Section 1) and **Polymorphism**, with **Low Coupling** — F7 depends only on `IMazeQuery` for collision.

Dependencies. `IMazeQuery` from F2 for wall collision. Nothing else. F7 does not know about hazards, puzzles, or the camera; `Kill` is called *into* it by F1.

Testing strategy. Every `InputTransformer` is a pure function over `Vector2` — assert `MirrorTransformer.Transform((1,0)) == (-1,0)` and so on, including stacked compositions. Collision is tested against a hand-built 3×3 stub `IMazeQuery` with no Unity scene. Boundary tests: zero input vector (must survive every transformer); diagonal input at exactly 45° (normalisation must not let diagonal movement outrun cardinal). Stress test: 1,000 input samples per frame through a 10-deep transformer stack.

Completion criteria. The mouse moves on WASD, never clips a wall, all four control modifiers work individually and stacked, death and respawn are clean, and the input layer has at least one non-keyboard binding proving the abstraction holds.

Effort (~52h). Design 9 · Coding 23 · Testing 12 · Documentation 8. The optional second-player controller is *not* included in this 52 and must not begin before Week 10.

4. Architectural overview

One paragraph per feature — enough context to write a Champion document without needing implementation details from anyone else.

F1 — Game Session & Level Flow. Sits at the centre and holds every other feature at arm's length. Each frame it reads the player's cell from F7, pushes it to F3 and F4, asks F3 whether the objectives are complete and F4 whether the mouse has been caught, ticks the timer, and runs its rule list to decide whether the level has ended. On level start it asks F2 for a maze, hands the anchor points to F3 and F4, hands the maze bounds and the player transform to F5, and tells F6 what to display. Nothing calls back into F1 except through events. If you are writing any other Champion document, the only thing you need to know about F1 is that it will call your public methods and subscribe to your events — it will never reach inside you, and you will never reach inside it.

F2 — Maze Generation & Level Data. The foundation everything else stands on and the only feature with no upstream dependency. It answers three kinds of question: *what does the maze look like* (for F5's framing and minimap), *is this cell walkable* (for F7's collision and F4's pathfinding), and *give me somewhere to put a thing* (for F3's puzzles and F4's hazards). Anchor reservation is the key idea — F3 and F4 get cells to build on without either of them understanding maze topology, and without any chance of them colliding. Because F2 is seeded and deterministic, every other developer can pin a seed in their tests and get the same maze forever.

F3 — Puzzle & Objective System. Owns the win condition. It builds a tree of puzzle nodes onto anchors at level start, receives the player's cell each frame from F1, and raises an event when the tree evaluates to complete. It has no knowledge of the exit door, the timer, hazards, or the camera — it answers one question, and F1 decides what the answer means. If you are working on another feature, F3 affects you only through the HUD progress string and the moment the exit unlocks.

F4 — Hazards & Threats. Owns the loss condition that isn't the clock. It populates hazards onto anchors, advances them every tick, and reports when the mouse's cell is lethal. The predator is the one hazard that needs the maze — it pathfinds through `IMazeQuery`. Everything else it needs arrives as a `Vector2Int` per frame. Crucially, F4 never kills the player directly: it raises `OnPlayerCaught`, F1 arbitrates, and F1 calls `Kill` on F7. That indirection is deliberate, and it is what lets F4 be tested with no player in the scene at all.

F5 — Perception, Camera & Visual Distortion. Purely downstream. It receives a transform to follow and maze bounds at level start, a distortion list from F1's level config, and nothing else per frame. It cannot affect gameplay — if F5 were deleted entirely, the game would still be winnable, just unwatchable. That one-way relationship is what makes it safe to develop in parallel with everything else and safe to iterate on visually right up to Week 12.

F6 — HUD, Menus & Audio. The only feature with a purely presentational role, and the only one that talks exclusively to F1. It receives a model of the current state and renders it; it raises button events and F1 decides what they mean. It has no reference to the maze, the player, the puzzles or the hazards. For other developers, F6 matters in one way: if your feature has state a player needs to know about, expose it as a value on your interface and F1 will route it to the HUD. Do not draw it yourself.

F7 — Player Character & Input. The mouse, and the only feature that reads the keyboard. Everything above the raw device is transformation: the control distortion stack is what makes level 9 hard, and it lives here rather than anywhere else because it is *input* being changed, not the world. F7 exposes its transform for F5 to follow and its cell for F1 to distribute, and it receives `Kill` calls rather than making them. It depends on F2 for collision and on nobody else.

5. MVP core

TL1 should build the MVP around F2 — Maze Generation & Level Data (TL2).

This is what A5 requires, and it is also the right answer on the merits:

- **It is the only feature with no dependencies.** F2 compiles and runs with every other feature absent. Nothing else in the project can say that — F3 and F4 need anchors, F5 needs a follow target, F6 needs a state model, F7 needs walkability, and F1 needs all of them.
- **It is the feature the game cannot fake.** A stubbed HUD is an ugly game; a stubbed maze is not a game at all. Everything else in *Algernon* is a modifier on top of "there is a maze and you are in it."
- **It de-risks the most consumers.** Six of seven features query F2. Getting its interface right in Week 4 is worth more than any other single decision the team makes.
- **It matches the timeline.** Section 6 puts the MVP at the TL2 capstone deadline, which is TL2's own lecture milestone.

What "playable MVP" means concretely. Per Section 6 the MVP must call at least one public method from every other feature and must survive their absence. The minimum shape:

Feature	Method TL1 calls in the MVP	If it isn't ready
F2	<code>Generate(config)</code> + <code>IsWalkable</code>	Blocking — no substitute
F7	<code>SpawnAt(spawnWorldPos)</code>	<code>NullPlayer</code> — camera free-flies
F3	<code>BuildPuzzles(config, anchors)</code>	<code>NullPuzzles</code> — exit always open
F4	<code>SpawnHazards(config, anchors)</code>	<code>NullHazards</code> — nothing kills you
F5	<code>BindFollowTarget(t)</code>	<code>NullView</code> — fixed top-down camera
F6	<code>SetTimer(seconds)</code>	<code>NullHud</code> — <code>Debug.Log</code> only

Every feature must ship its `Null*` stub in `Contracts/` in Week 4, before any real implementation. That is a one-hour job per developer and it is what makes the "MVP remains playable even if features are incomplete" requirement true rather than aspirational.

6. How the features interact — coupling, cohesion, and GRASP

The architectural spine

There is one rule, and it produces everything else:

State is pushed down by F1. Events are published up to F1. No feature ever holds a reference to another feature's concrete type.

Every cross-feature interaction in *Algernon* is one of three shapes:

1. **Level-start handover** — F1 gets something from one feature and gives it to another (maze anchors → puzzles and hazards; player transform → camera).
2. **Per-frame push** — F1 reads one small value and pushes it to whoever needs it (`playerCell` → F3 and F4).
3. **Event publication** — a feature raises an event and F1 decides what it means (`OnExitUnlocked` , `OnPlayerCaught` , `OnRetryPressed`).

None of these requires a feature to reference another feature. The only types crossing directories are interfaces, enums, and small immutable structs in `Contracts/` . This is why no developer will ever need to modify another developer's source code, which is the actual requirement in Section 3.

GRASP, principle by principle

Principle	Where it appears	Why it is the right call
Information Expert	<code>MazeData</code> answers every maze question (F2); each <code>PuzzleElement</code> evaluates its own completion (F3); each <code>Hazard</code> knows what it threatens (F4)	The object holding the data does the work. The alternative — F1 computing puzzle completion — would put F3's logic in F1's directory, which Section 4 forbids.
Creator	<code>MazeAlgorithm</code> creates <code>Cell</code> (F2); <code>PuzzleFactory</code> creates elements from anchors (F3); <code>HazardScheduler</code> creates hazards (F4)	Each creator has the initialising data and closely uses what it creates.
Controller	<code>GameCoordinator</code> (F1) is the single system-event handler for the run	Seven features and one coordinator. The alternative — features triggering each other — is the coupling this project is explicitly designed to avoid.
Low Coupling	No feature references another's concrete types; F4 needs a <code>Vector2Int</code> , F5 needs a <code>Transform</code> , F6 needs a <code>HudModel</code>	Measured by the ownership rule itself: if two developers can work for a week without merging, coupling is low enough.
High Cohesion	Each feature answers one question — <i>what does the maze look like / is it solved / what kills you / what can you see / what do you read / how do you move / what happens next</i>	The split was chosen on this criterion first: any feature whose one-line description needed an "and" was re-split.
Polymorphism	Seven hierarchies: <code>LevelRule</code> , <code>MazeAlgorithm</code> , <code>PuzzleNode</code> , <code>Hazard</code> , <code>ViewEffect</code> , <code>HudWidget</code> , <code>InputTransformer</code>	Variation is handled by type everywhere; there is no type-code switch in the design, which is the point of the Section 7 requirement.
Pure Fabrication	<code>SessionMachine</code> (F1), <code>AnchorRegistry</code> (F2), <code>ObjectiveTree</code> (F3), <code>HazardScheduler</code> and <code>GridPathfinder</code> (F4), <code>ViewEffectPipeline</code> and <code>FogGrid</code> (F5), <code>ScreenRouter</code> (F6)	None is a thing in the game world. Each holds behaviour that would otherwise pollute a domain class or leak across a feature boundary.
Indirection	F1 mediates every cross-feature interaction; <code>ObjectiveTree</code> sits between puzzles and the exit; F4 raises an event rather than calling <code>Kill</code>	Indirection is the cost the project pays for the ownership rules. It is worth naming that trade-off in the post-mortem.
Protected Variations	<code>IMazeQuery</code> (F2's algorithms can change freely), <code>IInputSource</code> (F7 can add touch and WebGL without touching movement), <code>HudFacade</code> (F6's widgets can be rewritten)	These are the three places the project <i>knows</i> variation is coming: new maze algorithms, new platforms, and UI iteration.

The honest weaknesses

Worth writing into the Week 14 post-mortem now, while they are still visible:

- **F1 is a hub.** Mediator concentrates knowledge in one place. It is right for these ownership constraints, but F1's file will be the longest in the project and TL1 becomes a bottleneck in Weeks 10–12.
- **Push-based state has a cost.** Pushing `playerCell` to F3 and F4 every frame is slightly wasteful compared to letting them query the player. The independence it buys is worth it, but it is a trade, not a free win.
- **Indirection can hide bugs.** When F4 raises `OnPlayerCaught` and nothing happens, the fault could be in F4, in F1's subscription, or in F1's arbitration. Log every event crossing F1 from Week 4.

7. Initial Gantt chart — Weeks 4 to 12

Week 4 coding begins (repo live, Champions approved) · Week 5 SA presentations, reduced capacity · Week 6 class diagrams and test plan, design-heavy · Week 8 midterm, reduced capacity · Week 12 last full coding week, code freezes at the start of Week 13.

Weeks 4–8 — build-out

Dev	Week 4	Week 5	Week 6	Week 7	Week 8
TL1	Contracts, stubs, scene bootstrap	Session state machine	Class diagram, integration tests	Timer, win/loss rules	<i>Midterm — stabilise loop</i>
TL2	Grid model, backtracker carve	Seeded generation, connectivity	Class diagram, boundary tests	Braiding, difficulty scaling	<i>Midterm — anchor API polish</i>
TL2+	Puzzle base, pressure plate	Composite objective tree	Class diagram, unit tests	Lever and key puzzles	<i>Midterm — progress reporting</i>
TL3	Hazard base, static spikes	Timed trap scheduling	Class diagram, stress tests	Predator State machine	<i>Midterm — telegraph cues</i>
TL4	Camera rig, follow target	Fog of war reveal	Class diagram, effect tests	Mirror and rotation effects	<i>Midterm — assemble chain</i>
TL5	HUD skeleton, timer widget	Main menu, pause screen	Class diagram, widget tests	Objective and result screens	<i>Midterm — audio cue table</i>
TL6	Mouse movement, wall collision	Input abstraction layer	Class diagram, input tests	Mirror/invert decorators	<i>Midterm — death and respawn</i>

Weeks 9–12 — integration and freeze

Dev	Week 9	Week 10	Week 11	Week 12
TL1	Level progression, statistics	Full seven-feature integration	CI/CD green, integration hardening	Freeze prep, demo script
TL2	Tilemap build, world mapping	Performance tuning, large mazes	Edge-case bugfix	Docs, oral rehearsal
TL2+	Push-block, sequence puzzle	Reset and retry correctness	Balance, integration bugfix	Docs, oral rehearsal
TL3	Flood hazard, difficulty scaling	Kill arbitration integration	Balance passes, bugfix	Docs, oral rehearsal
TL4	Minimap, blackout effect	Per-level distortion scheduling	Performance, visual polish	Docs, oral rehearsal
TL5	Level intro and outro flow	Settings, accessibility options	Feedback and polish pass	Docs, oral rehearsal
TL6	Animation states, feel tuning	<i>Optional</i> second-player controller	WebGL and mobile input mapping	Docs, oral rehearsal

Notes for TL4 when importing into GitHub Projects. Week 6 and Week 8 are deliberately light on new code — Week 6 because the class diagram gate [G4] and the initial test plan (2 boundary + 1 stress per feature, all passing) consume most of the week, and Week 8 because of the midterm. Week 10 is the integration crunch: every feature must be callable from F1 by the end of it, because Week 11 is CI/CD and Week 12 is the last chance to fix anything before the freeze. TL6's Week 10 cell is the only optional work in the chart and should be the first thing cut if anything slips.

8. Risks and mitigations

Ordered by how early they will hurt.

R1 · The MVP deadline predates the coding gate. Section 6 puts the MVP at the end of Week 3; Gate G2 permits coding in Week 4. These cannot both be satisfied literally.

→ *Mitigation:* TL1 raises this with the instructor in Week 3, not Week 4. Propose that the Week 3 deliverable is the TL2 capstone plus a bootstrap scene with seven `Null*` stubs, and that "MVP" in the Section 6 sense lands end of Week 4. Get the answer in writing.

R2 · `Contracts/` may not survive the ownership rules. The entire architecture assumes a shared folder of interfaces and enums. Section 4 says shared implementation code is not permitted and only the owner may modify their directories.

→ *Mitigation:* Confirm with the instructor in Week 3 that interfaces, enums and immutable structs are not "implementation". If the answer is no, the fallback is duplicated interface definitions per feature plus adapter classes in TL1's directory — significantly worse, and better discovered now than in Week 6.

R3 · F2 is on the critical path for six features. If maze generation slips, everyone slips.

→ *Mitigation:* Freeze the `IMazeQuery` interface on day 1 of Week 4, before any generation code exists. TL2 ships a hand-authored `FixedTestMaze` implementing it in the first two days. Every other developer builds and tests against `FixedTestMaze` and never blocks on the real generator.

R4 · The Unity scene file is the one thing everybody has to touch. Scene `.unity` files merge badly and are the classic way a code-ownership rule gets violated by accident.

→ *Mitigation:* One thin bootstrap scene, owned by TL1 exclusively. Every feature ships a prefab in `prefabs/TL#_name/` that TL1 instantiates at runtime. Configure Unity Smart Merge (`git config merge.unityyamlmerge`) in Week 4 and set `*.unity merge=unityyamlmerge` in `.gitattributes` before the first commit.

R5 · Hazard and puzzle placement can make a level unsolvable. A guillotine on the only corridor, or a lever behind a locked door that needs the lever.

→ *Mitigation:* Two layers. F2's `ReserveAnchors` never hands the same cell out twice and never returns the spawn or exit cell. F1 runs a BFS solvability assertion after placement and regenerates on failure. Make this an automated test: 1,000 seeds at every difficulty, assert 100% solvable.

R6 · The dynamic binding demos can quietly become undemonstrable. Several of the virtual methods above are only meaningful if they are called through a base-typed reference. A developer who refactors to a concrete type has silently destroyed their oral exam answer without breaking anything.

→ *Mitigation:* Register each virtual/override pair at Gate G4 in Week 6 and add a unit test per feature that calls the virtual method through the base type and asserts the derived behaviour. If someone breaks the polymorphism, the test goes red the same day.

R7 · The `virtual`-removal demo does not compile the way people expect. Removing `virtual` from the base while leaving `override` on the derived class is a compile error in C# (CS0506), not a behaviour change. The demo requires changing `override` to `new` at the same time.

→ *Mitigation:* Every developer rehearses the exact two-keystroke edit before Week 12 and keeps it on a `demo/no-virtual` branch. Nobody discovers this at the oral exam.

R8 · Two features both implement "mirroring". TL6 mirrors input, TL4 mirrors rendering. Overlap here means duplicated work or, worse, both applying at once and cancelling out.

→ *Mitigation:* Written contract in both Champion documents: TL6 transforms `Vector2` input only, TL4 transforms the view matrix only, and F1's `LevelConfig` names exactly which of the two (or both) each level uses. Add a level to the test plan that uses only one, and one that uses both.

R9 · Effort is unlikely to land evenly at 52 hours. F4's predator AI and F5's rendering pipeline are the most likely to overrun; F6 is the most likely to finish early.

→ *Mitigation:* Maintain a named backlog of transferable stretch items — additional puzzle types, additional hazards, extra distortions, accessibility options — each pre-assigned to a *second* owner. The Week 7 Roast is the checkpoint where reallocation actually happens.

R10 · TL1 becomes a bottleneck in Weeks 10–12. One integrator, six features landing at once.

→ *Mitigation:* Integration is weekly from Week 4, not a Week 10 event. Every feature must run in its own test scene without F1, so a broken feature never blocks the others. TL1's own feature work is front-loaded into Weeks 4–7 for exactly this reason.

R11 · Optional scope creep from A4's second player controller. A co-op level is genuinely appealing and would consume TL6's entire buffer.

→ *Mitigation:* It is explicitly outside TL6's 52 hours, cannot begin before Week 10, and is the first item cut. Do not put it in the Champion document as a commitment — put it in as a stretch goal with a named cut date.

R12 · Gate G6 (Week 10) audits the AI log. This plan is AI-generated material that shaped the entire architecture.

→ *Mitigation:* TL5 logs this document — prompt, date, model, and what was changed by hand afterwards — in the AI log on the day it is generated, and every Champion document cites it as its source. Retrofitting an audit trail in Week 10 is much worse than keeping one from Week 3.

Appendix — what to do next

1. **TL5:** log this document in the AI log today; circulate it before the Champions begin.
2. **All seven:** read your own Section 3 entry and the Section 4 paragraph for the two features nearest yours. Write your Champion document from your Section 3 entry — it is the contract for what you will build.
3. **TL1:** take R1 and R2 to the instructor this week. Both are answerable in five minutes and both are expensive to discover late.
4. **TL2:** freeze `IMazeQuery` and ship `FixedTestMaze` in the first two days of Week 4. Six people are waiting on it.
5. **TL4:** import the Section 7 Gantt into GitHub Projects; the Week 6 and Week 8 rows are deliberately light and should stay that way.
6. **Everyone, before Week 6:** register your virtual/override pair and write the test that calls it through the base type.

Generated from the CS3383 TL5 AI Planning Prompt. Section A supplied by the team at the Meet and Greet; Section B is fixed course requirements.